

Getting Started with ROS 2 Simulators

Summary

1. Why using simulators?
2. Introduction to Gazebo
3. Simple example: mobile robot
4. Advanced example: multi drones
5. Useful links and resources

Why using simulators?

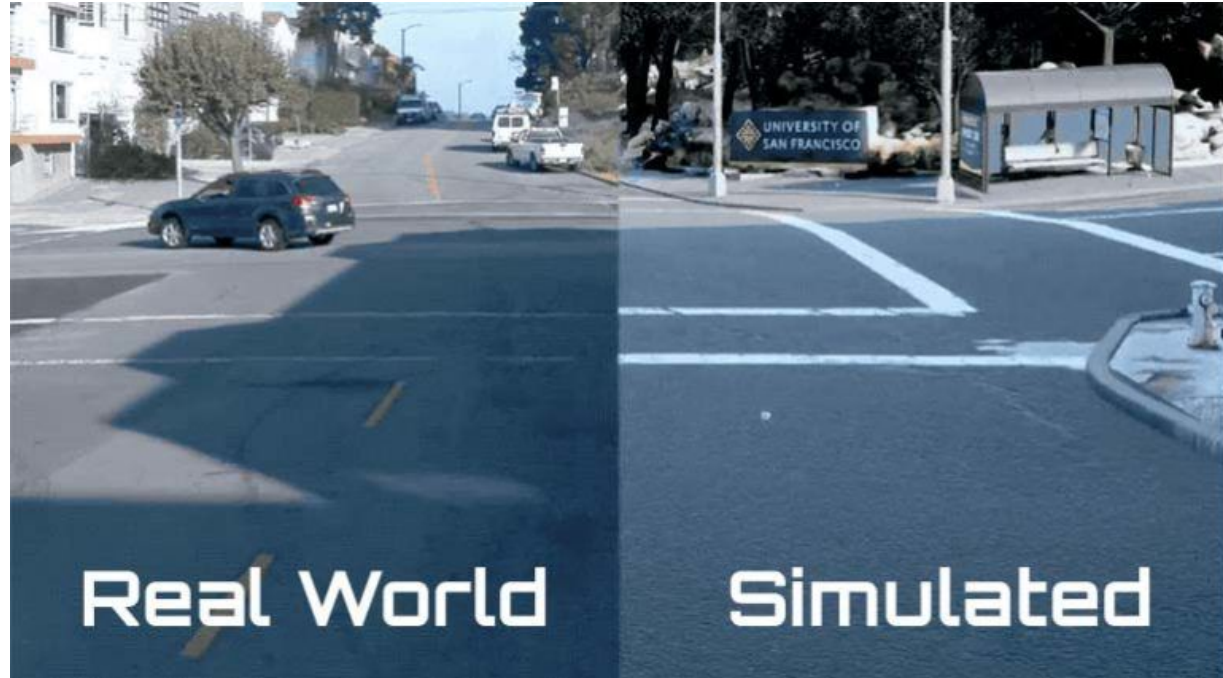
Why using simulators?

Why using a simulator?



Why using a simulator?

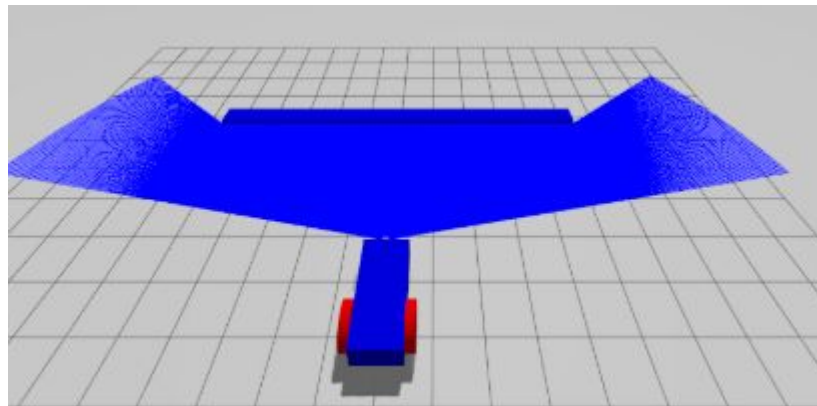
- Safety reasons
- Reducing costs
- Quick proof of concept
- Proving repeatability
- Quality of the code



What is a robotic simulator?

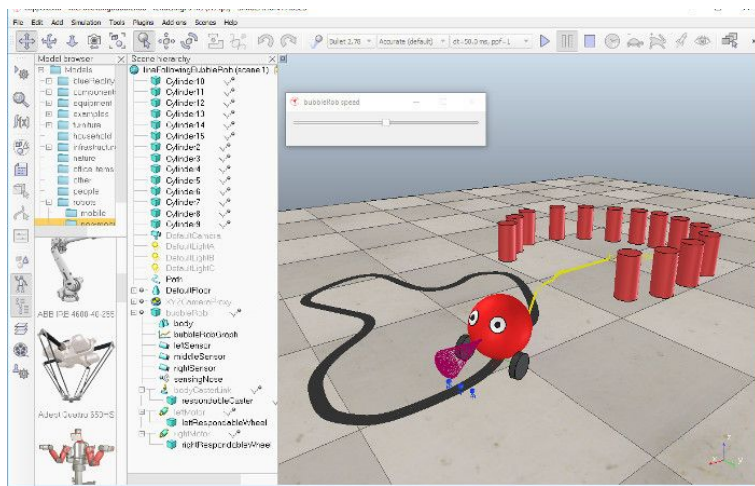
A robotic simulator usually contains:

- Simulated sensors
- Simulated controllers
- Realistic 3d rendering
- A physics engines
- A GUI



Benchmark of the robotic simulators

- Webots
- Gazebo
- NVIDIA Isaac Sim
- CoppeliaSim



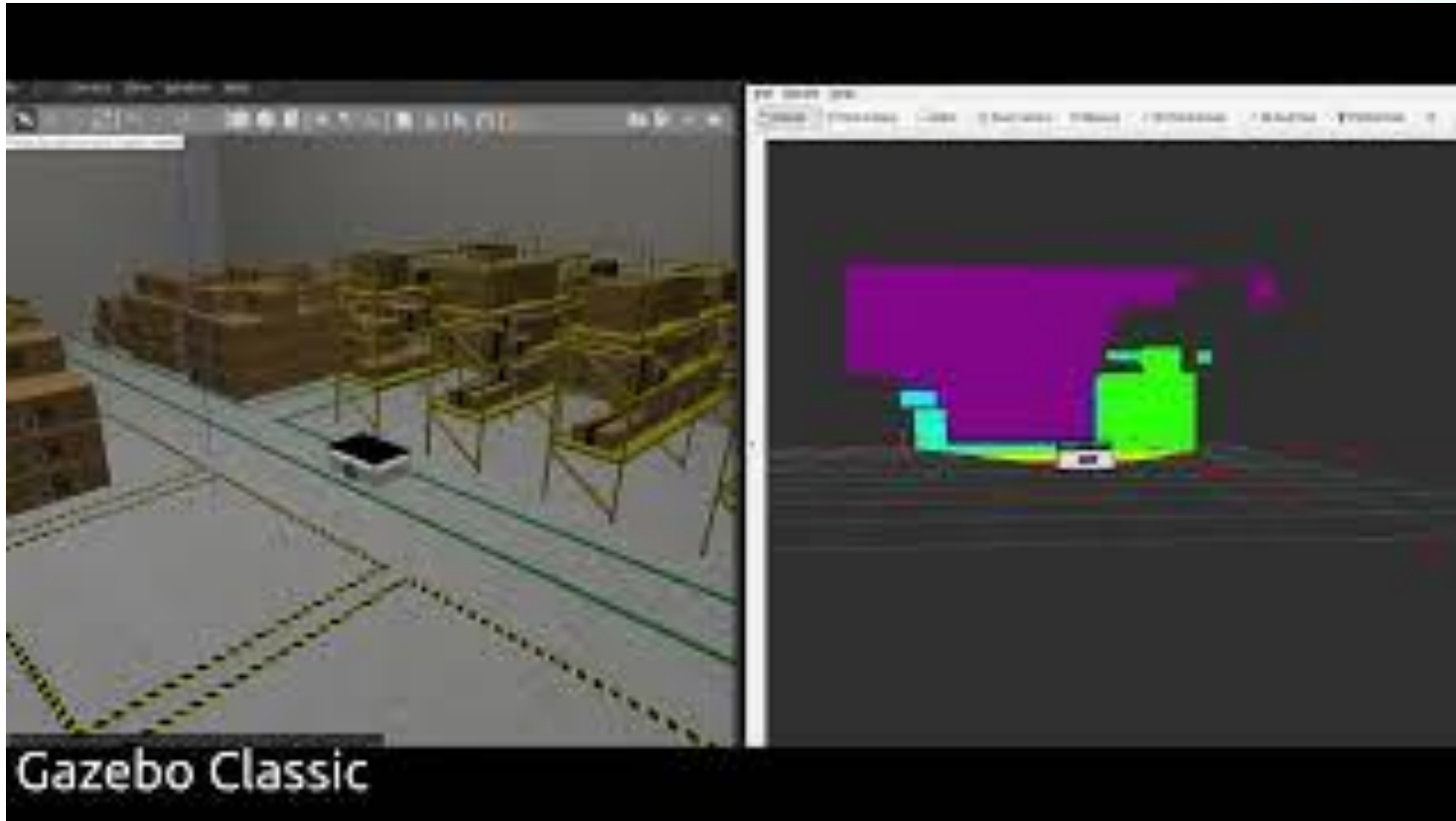
Introduction to Gazebo

Why did we choose Gazebo?

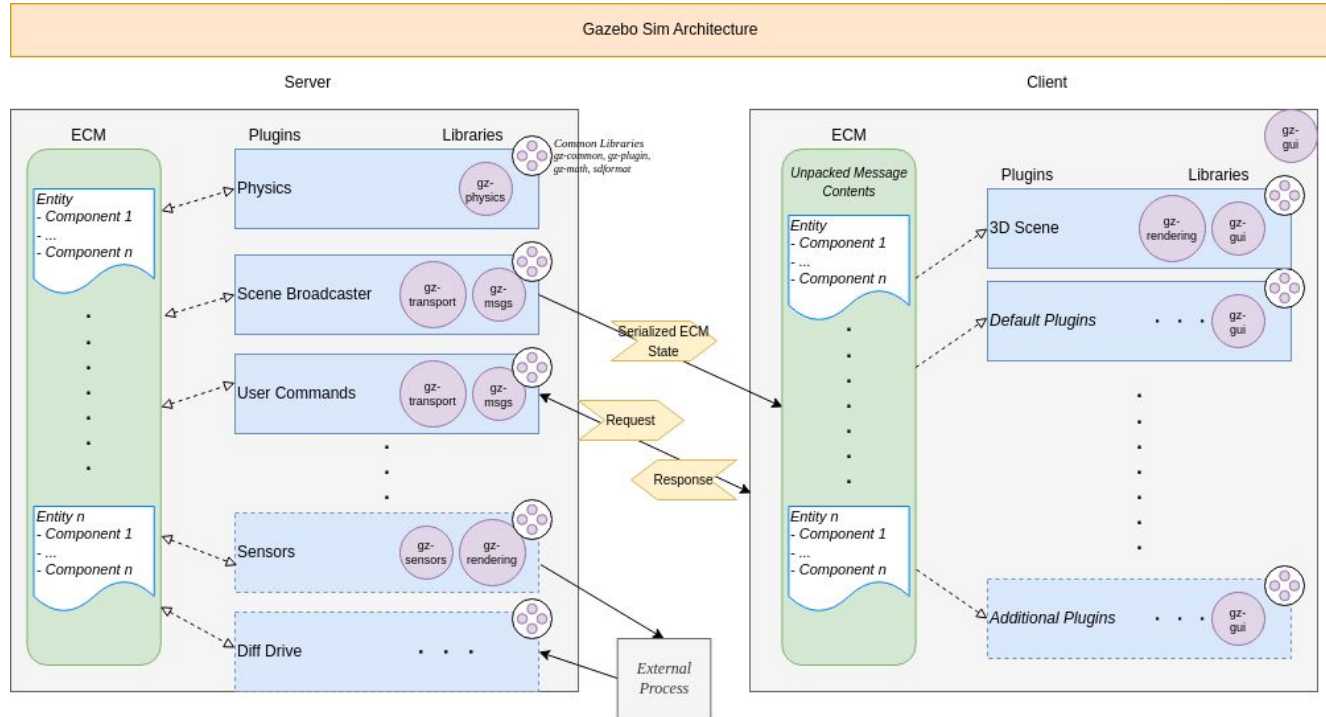
- **Part of the ROS PPA**
- **Well documented / supported**
- **Wide community**
- **Has issues but is sufficiently good**
- **Open source**



What does it look like?



Introduction to Gazebo Fortress



Note: a gazebo simulation can run without ROS!

Introduction to Gazebo Fortress

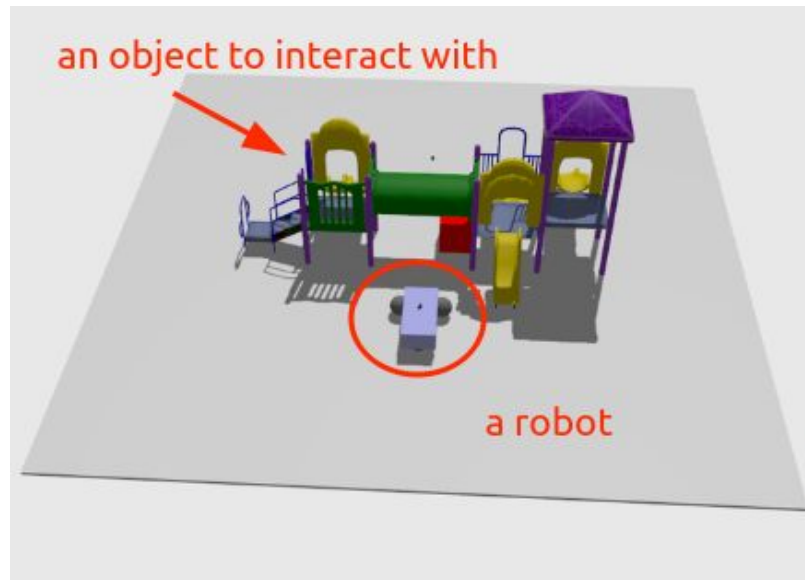
A gazebo simulation can be customised using:

- **world files (sdf):** *define worlds and their properties*
- **models files (sdf):** *define the entities to be spawned in the world*
- **plugins files (cpp):** *can be of different types (model, world, sensor, visual, ...)*

Simple example: mobile robot

What we want

- **A mobile robot with a lidar**
→ *Defining sensor plugin and sdf model*
- **An object to populate the world**
→ *Defining a sdf model and a sdf world*
- **A plugin to teleoperate the robot**
→ *Defining a control/world plugin*



Defining and importing a plugin

1. Implementation in cpp

2. Imported in a sdf

```
<plugin  
  filename="gz-sim-sensors-system"  
  name="gz::sim::systems::Sensors">  
  <render_engine>ogre2</render_engine>  
</plugin>
```

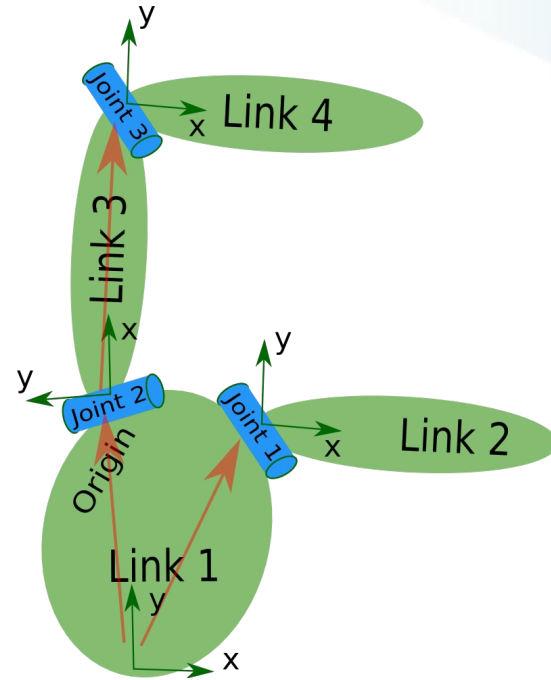
3. Configuration in a sdf

```
<sensor name='gpu_lidar' type='gpu_lidar'>"  
  <pose relative_to='lidar_frame'>0 0 0 0 0 0</pose>  
  <topic>lidar</topic>  
  <update_rate>10</update_rate>  
  <ray>  
    <scan>  
      <horizontal>  
        <samples>640</samples>  
        <resolution>1</resolution>  
        <min_angle>-1.396263</min_angle>  
        <max_angle>1.396263</max_angle>  
      </horizontal>  
      <vertical>  
        <samples>1</samples>  
        <resolution>0.01</resolution>  
        <min_angle>0</min_angle>  
        <max_angle>0</max_angle>  
      </vertical>  
    </scan>  
    <range>  
      <min>0.08</min>  
      <max>10.0</max>  
      <resolution>0.01</resolution>  
    </range>  
  </ray>  
  <always_on>1</always_on>  
  <visualize>true</visualize>  
</sensor>
```

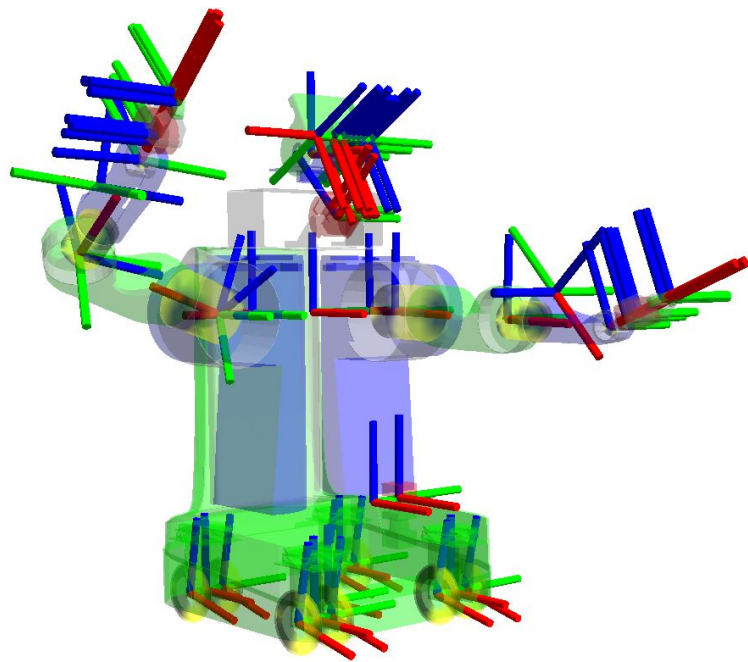
Creating a model

A model file usually contains:

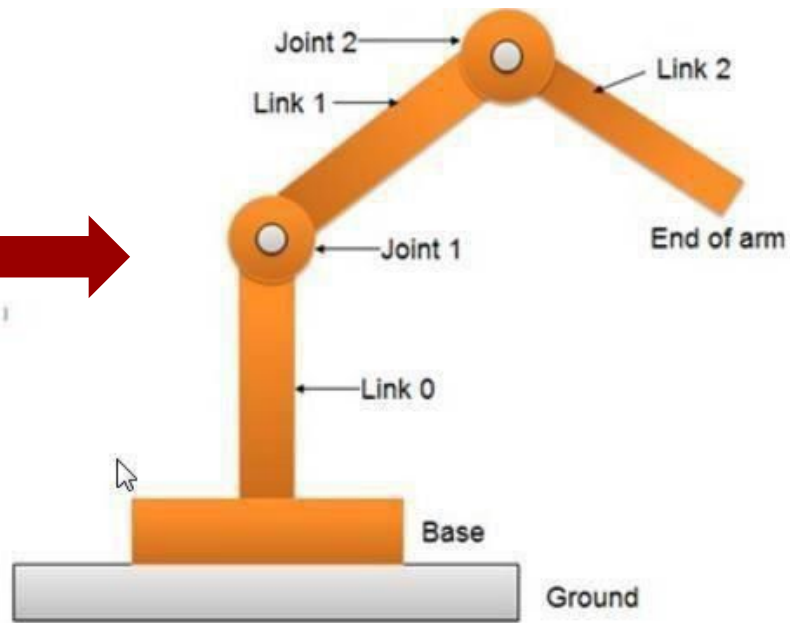
- **links:** *defines a rigid body entity*
- **joints:** *describes the kinematics and dynamics of the joint*



Reminder: Frame representation in ROS2



Tfs in ros



Links and joints in gazebo

Creating a Link

```
<!--chassis-->
<link name='chassis'>
  <pose relative to=' model ' >0.5 0 0.4 0 0 0</pose>
  <inertial> <!--inertial properties of the link mass, inertia matix-->
    <mass>1.14395</mass>
    <inertia>
      <ixx>0.095329</ixx>
      <ixy>0</ixy>
      <ixz>0</ixz>
      <iyy>0.381317</iyy>
      <iyz>0</iyz>
      <izz>0.476646</izz>
    </inertia>
  </inertial>
  <visual name='visual'>
    <geometry>
      <box>
        <size>2.0 1.0 0.5</size>
      </box>
    </geometry>
    <!--Color of the link-->
    <material>
      <ambient>0.0 0.0 1.0 1</ambient>
      <diffuse>0.0 0.0 1.0 1</diffuse>
      <specular>0.0 0.0 1.0 1</specular>
    </material>
  </visual>
  <collision name='collision'>
    <geometry>
      <box>
        <size>2.0 1.0 0.5</size>
      </box>
    </geometry>
  </collision>
</link>
```

Relative pose

Inertial
properties

Rendering
properties

collision
properties

```
world name="visualize_lidar_world">  
<model name='vehicle blue'>
```

```
<link name="lidar_link">  
  <pose>0 0 0.5 0 0 0</pose>  
  <inertial>  
    <mass>0.1</mass>  
    <inertia>  
      <ixx>0.000166667</ixx>  
      <iyy>0.000166667</iyy>  
      <izz>0.000166667</izz>  
    </inertia>  
  </inertial>  
  <collision name="collision">  
    <geometry>  
      <box>  
        <size>0.1 0.1 0.1</size>  
      </box>  
    </geometry>  
  </collision>  
  <visual name="visual">  
    <geometry>  
      <box>  
        <size>0.1 0.1 0.1</size>  
      </box>  
    </geometry>  
  </visual>
```

```
<sensor name='gpu_lidar' type='gpu_lidar'>  
  <pose>0 0 0 0 0 0</pose>  
  <topic>lidar2</topic>  
  <update_rate>10</update_rate>  
  <lidar>  
    <scan>  
      <horizontal>  
        <samples>640</samples>  
        <resolution>1</resolution>  
        <min_angle>-1.396263</min_angle>  
        <max_angle>1.396263</max_angle>  
      </horizontal>  
      <vertical>  
        <samples>1</samples>  
        <resolution>1</resolution>  
        <min_angle>0.0</min_angle>  
        <max_angle>0.0</max_angle>  
      </vertical>  
    </scan>  
    <range>  
      <min>0.08</min>  
      <max>10.0</max>  
      <resolution>0.01</resolution>  
    </range>  
  </lidar>  
  <visualize>true</visualize>  
</sensor>
```

Customizing a Link

Sensors plugins are added and configured directly **INSIDE** the link!

Creating a joint

! Make sure the links exist before creating the joint!

```
<!--left wheel joint-->
<joint name='left_wheel_joint' type='revolute'>
  <pose relative_to='left_wheel' />
  <parent>chassis</parent>
  <child>left_wheel</child>
  <axis>
    <xyz expressed_in='__model__'>0 1 0</xyz> <!--can be descired to any frame or even arbitrary frames-->
    <limit>
      <lower>-1.79769e+308</lower>    <!--negative infinity-->
      <upper>1.79769e+308</upper>    <!--positive infinity-->
    </limit>
  </axis>
</joint>
```

Model file

```
<model name='vehicle_blue'>
  <pose>-4 0 0.325 0 0 0.0</pose>
  <link name='chassis'>...
</link>

  <link name='lidar_link'>...
</link>

  <link name='left_wheel'>...
</link>

  <link name='right_wheel'>...
</link>

  <link name='caster'>...
</link>

  <joint name='lidar_sensor_joint' type='fixed'>...
</joint>

  <joint name='left_wheel_joint' type='revolute'>...
</joint>

  <joint name='right_wheel_joint' type='revolute'>...
</joint>

  <joint name='caster_wheel' type='ball'>...
</joint>

  <plugin
    filename="gz-sim-diff-drive-system"
    name="gz::sim::systems::DiffDrive">
    <left_joint>left_wheel_joint</left_joint>
    <right_joint>right_wheel_joint</right_joint>
    <wheel_separation>1.25</wheel_separation>
    <wheel_radius>0.3</wheel_radius>
    <odom_publish_frequency>1</odom_publish_frequency>
  </plugin>
</model>
```

Models can be inserted
explicitly or imported
from [fuel](https://fuel.gazebosim.org/1.0/openrobotics/models/Playground) or locally

```
<include>
  <pose>0 0 0 0 0 1.57</pose>
  <uri>https://fuel.gazebosim.org/1.0/openrobotics/models/Playground</uri>
</include>
```


World file

```
<?xml version="1.0" ?>
<!--
  You can echo lidar messages using:
  gz topic -e -t /lidar
-->
<sdf version="1.10">
  <world name="visualize lidar world">
    <physics name="lms" type="ignored">
      <max_step_size>0.001</max_step_size>
      <real_time_factor>1.0</real_time_factor>
    </physics>
    <plugin
      filename="gz-sim-physics-system"
      name="gz::sim::systems::Physics">
    </plugin>
    <plugin
      filename="gz-sim-sensors-system"
      name="gz::sim::systems::Sensors">
      <render_engine>ogre2</render_engine>
    </plugin>
    <plugin
      filename="gz-sim-scene-broadcaster-system"
      name="gz::sim::systems::SceneBroadcaster">
    </plugin>

    <gui fullscreen="0">...
    </gui>

    <light type="directional" name="sun">...
    </light>

    <model name="ground_plane">...
    </model>

    <model name="box">...
    </model>

    <model name="model_with_lidar">...
    </model>

    <model name='vehicle_blue'>...
    </model>

    <include>
      <pose>0 0 0 0 1.57</pose>
      <uri>https://fuel.gazebosim.org/1.0/openrobotics/models/Playground</uri>
    </include>

  </world>
</sdf>
```

! Simulated time is not real time!

- **max_step_size**: *~ time step size*
- **real_time_factor**: *how many simulation steps per real second you aim for*

Important exports

Environment variables:

`IGN_GAZEBO_RESOURCE_PATH` Colon separated paths used to locate resources such as worlds and models.

`GZ_SIM_RESOURCE_PATH` Colon separated paths used to locate resources such as worlds and models.

`IGN_GAZEBO_SYSTEM_PLUGIN_PATH` Colon separated paths used to locate system plugins.

`GZ_SIM_SYSTEM_PLUGIN_PATH` Colon separated paths used to locate system plugins.

`IGN_GAZEBO_SERVER_CONFIG_PATH` Path to server configuration file.

`GZ_SIM_SERVER_CONFIG_PATH` Path to server configuration file.

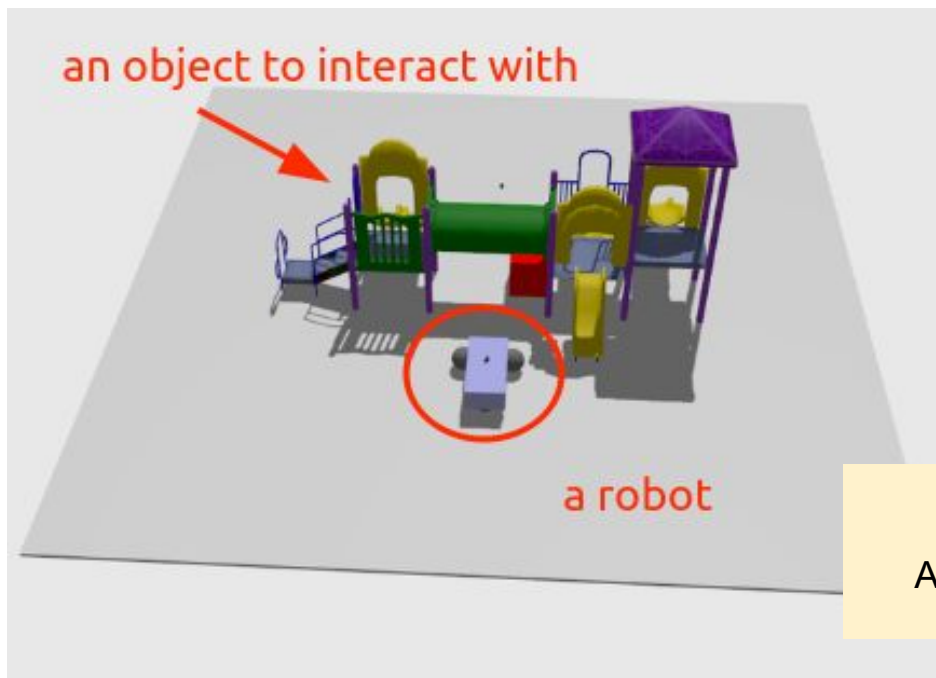
`IGN_GUI_PLUGIN_PATH` Colon separated paths used to locate GUI plugins.

`GZ_GUI_RESOURCE_PATH` Colon separated paths used to locate GUI resources such as configuration files.

Launching gazebo standalone

Gazebo standalone

```
ign gazebo -v 4 -r visualize_lidar.sdf
```



MISSION
ACCOMPLISHED!

Bridging between gazebo and ROS2

- Required to map gazebo topics to ros2 topics
- The topic type has to be given
- Mapping can be uni or bidirectional
- Can be done from a command line

```
ros2 run ros_gz_bridge parameter_bridge /keyboard/keypress@std_msgs/msg/Int32@ignition.msgs.Int32
```

or a yaml file:

```
# Diff drive configuration.  
- topic_name: "/model/vehicle_blue/cmd_vel"  
  ros_type_name: "geometry_msgs/msg/Twist"  
  gz_type_name: "gz.msgs.Twist"  
  lazy: true  
  direction: ROS_TO_GZ
```

```
- topic_name: "/model/vehicle_blue/odometry"  
  ros_type_name: "nav_msgs/msg/Odometry"  
  gz_type_name: "gz.msgs.Odometry"  
  lazy: true  
  direction: GZ_TO_ROS
```

Launching gazebo and ROS2

loading the
robot model

loading the
world

publishing the
robot frames

bridging
between
gazebo and ros

```
def generate_launch_description():
    # Setup project paths
    pkg_project_bringup = get_package_share_directory('ros_gz_example_bringup')
    pkg_project_gazebo = get_package_share_directory('ros_gz_example_gazebo')
    pkg_project_description = get_package_share_directory('ros_gz_example_description')
    pkg_ros_gz_sim = get_package_share_directory('ros_gz_sim')

    # Load the SDF file from "description" package
    sdf_file = os.path.join(pkg_project_description, 'models', 'diff_drive', 'model.sdf')
    with open(sdf_file, 'r') as infp:
        robot_desc = infp.read()

    # Setup to launch the simulator and Gazebo world
    gz_sim = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(pkg_ros_gz_sim, 'launch', 'gz_sim.launch.py')),
        launch_arguments={'gz_args': PathJoinSubstitution([
            pkg_project_gazebo,
            'worlds',
            'diff_drive.sdf'
        ]).items(),
    })

    # Takes the description and joint angles as inputs and publishes the 3D poses of the robot links
    robot_state_publisher = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        name='robot_state_publisher',
        output='both',
        parameters=[
            {'use_sim_time': True},
            {'robot_description': robot_desc},
        ]
    )

    # Visualize in RViz
    rviz = Node(
        package='rviz2',
        executable='rviz2',
        arguments=['-d', os.path.join(pkg_project_bringup, 'config', 'diff_drive.rviz')],
        condition=IfCondition(LaunchConfiguration('rviz'))
    )

    # Bridge ROS topics and Gazebo messages for establishing communication
    bridge = Node(
        package='ros_gz_bridge',
        executable='parameter_bridge',
        parameters=[{
            'config_file': os.path.join(pkg_project_bringup, 'config', 'ros_gz_example_bridge.yaml'),
            'qos_overrides./tf_static.publisher.durability': 'transient_local',
        }],
        output='screen'
    )
```

Useful links and resources

Useful links and resources

- [ROS2 humble documentation](#)
- [Gazebo Fortress documentation](#)
- [Simulator benchmark](#)
- [ROS2/Gazebo tutorials](#)
- [ROS2/Gazebo project template](#)
- [ROS2 and Gazebo best practices](#)
- [FUEL](#)

Questions?